

Talking Script: Software Architecture for Essential & Accidental Uncertainty

15-Minute Presentation Script with Timings

Total Time: 15 minutes (900 seconds)

Slide 1: Title Slide

Time: 0:00 - 0:15 (15 seconds)

Good morning. I'm George Rudolph from Utah Valley University in Orem, Utah, and I'm here today to present work by Neil Harrison, Peter Aldous and myself. Our paper is titled "Software Architecture for Essential & Accidental Uncertainty." This work addresses a critical gap in how we think about uncertainty in software systems, and provides a framework for architects to make better design decisions.

Slide 2: Introduction: A Bit of History

Time: 0:15 - 0:50 (35 seconds)

Let me start with a bit of history. The ENIAC, shown here, was the first general-purpose computer, built between 1945 and 1955. It was designed to solve a large class of numerical problems. This was the first computer to run at electronic speed, though it was unfortunately killed by a lightning strike in 1955.

What's important for our discussion is that the early math problems it computed were precisely specified. We had well-defined inputs, clear algorithms, and deterministic outputs. The results were computed deterministically—there was no ambiguity about what the system should do or what the correct answer was.

Slide 3: Progress

Time: 0:50 - 1:20 (30 seconds)

But computing has evolved dramatically since then. Original computing tackled precisely-defined mathematical problems. Over time, capabilities expanded to business applications, data communication, control systems, and AI.

As this happened, inputs, outputs, and processes became more varied and less precise. This is an artifact of modeling non-mathematical problems via mathematical abstractions. We must now deal with uncertainty—and understand how to approach it in architecture and design.

Slide 4: Current State of the Art

Time: 1:20 - 1:45 (25 seconds)

In 2010, David Garlan called for uncertainty to be a first-class concern in software design and development. However, current research and practice primarily focus on fault tolerance and related tactics. While important, this misses a large part of uncertainty and motivates the deeper investigation we outline today.

Slide 5: Essence vs. Accident (Brooks, 1986)

Time: 1:45 - 2:25 (40 seconds)

Among other ideas, our work builds on a fundamental distinction made by Fred Brooks in his famous 1986 paper, “No Silver Bullet.” Brooks distinguished between accidental complexity—which is created by engineering choices and can be mitigated by better tools and abstractions—and essential complexity, which is inherent in the problem and cannot be removed.

We apply this same lens to uncertainty. Just as there is essential and accidental complexity, there is essential and accidental uncertainty. This distinction is crucial for making appropriate architectural decisions.

Slide 6: Essential and Accidental Uncertainty

Time: 2:25 - 2:50 (25 seconds)

Let me define these terms clearly. Accidental uncertainty can be separated from the problem—a transformation exists to remove it. Essential uncertainty, on the other hand, must be addressed as part of the problem. No transformation can remove it; it’s inherent to what we’re trying to accomplish.

Slide 7: Uncertainty and Data

Time: 2:50 - 3:15 (25 seconds)

Computing is fundamentally the processing of data. So we consider uncertainty across three dimensions: input, transformation, and output. Input uncertainty can be essential or accidental. The transformation from input to output can also be essential or accidental. Output uncertainty is often essential, and this is where things get architecturally interesting.

Slide 8: Input: Uncertainty Examples

Time: 3:15 - 3:55 (40 seconds)

Let's look at some examples of input uncertainty. Unreliable data transmission is accidental—data communication protocols implement transformations to remove this uncertainty. The amount of data is often accidental; we can handle it in subsets. User input errors are accidental; we parse for correctness and re-prompt.

But consider multiple concurrent access. This is accidental if handled discretely, but becomes essential if not. Think of a venue ticket website where multiple requests come in for the same seat simultaneously. The uncertainty about which request should succeed is essential to the problem domain.

Slide 9: Uncertainty in the Transformation (Algorithms)

Time: 3:55 - 4:50 (55 seconds)

The algorithm or processing logic that transforms input to output can introduce uncertainty. Let's break this down.

Accidental transformation uncertainty can be fixed. This includes hardware glitches, compiler differences, and implementation bugs. These are engineering problems with engineering solutions.

Essential transformation uncertainty is more fundamental. Sometimes no deterministic algorithm exists to compute the correct output. Image recognition is a classic example—there's no deterministic formula that says “this is a cat.”

Then there's essential but intractable uncertainty. The algorithm is computationally infeasible, so approximations introduce uncertainty in accuracy or precision. Examples include the game of Go or many optimization problems.

The critical insight here is that essential transformation uncertainty often reflects uncertain output—and this requires probabilistic or AI-based approaches.

Slide 10: Output Uncertainty

Time: 4:50 - 5:10 (20 seconds)

Accidental output uncertainty is generally less architecturally interesting—it's something we can fix. Essential output uncertainty, however, spans a wide spectrum and requires a corresponding spectrum of architectural approaches. This is where the real architectural challenge lies.

Slide 11: Essential Output Uncertainties

Time: 5:10 - 6:10 (60 seconds)

Let me walk you through the spectrum of essential output uncertainties. At one end, there is a known right answer—mathematical computation. Then, the right answer can be known—like asking how well a product sold on a sales website.

Moving along the spectrum, the right answer exists but is infeasible to compute directly—the game of Go has an astronomical state space. Then we have unknown right answers where we can guess—image recognition falls here.

Further along, the effect is unknown but can be measured—optimization problems. Then the effect is unknown and the system is chaotic—weather forecasting. There’s no specific answer, but we can observe the “right” direction—advertising and many simulation problems.

At the far end, there’s no single right answer because the problem is vague—automated prose writing. And finally, the problem itself is unknown—grounded theory.

This spectrum is crucial because different positions require fundamentally different architectural approaches.

Slide 12: Thought Question

Time: 6:10 - 6:40 (30 seconds)

Here’s a thought question: How would you characterize automatic software coding? The trick is that it depends on the nature of the software. If you cannot precisely specify what the generated software should do, how can you verify it? Generative AI makes good answers at once more vital and more complex. This is a real challenge we’re facing today.

Slide 13: Spectrum of Uncertainty

Time: 6:40 - 7:00 (20 seconds)

This diagram visualizes the spectrum we just discussed. As you can see, uncertainty ranges from deterministic problems with known answers all the way to problems where the problem itself is unknown.

Slide 14: Architectural Approaches

Time: 7:00 - 7:35 (35 seconds)

Different levels and characteristics of uncertainty suggest different architectural styles, patterns, and tactics. When there's virtually no uncertainty, batch processing works well. For accidental uncertainty, we can use pipes and filters, layers, or broker patterns.

For greater essential uncertainty, we need AI-based approaches: blackboard architectures, neural networks, machine learning, and deep learning. The key is matching the architecture to the nature of the uncertainty.

Slide 15: Spectrum of Architectural Approaches

Time: 7:35 - 7:50 (15 seconds)

This diagram maps architectural approaches to the uncertainty spectrum. As uncertainty increases, we move from traditional deterministic architectures to probabilistic and AI-based systems.

Slide 16: An Architecture Process for Uncertainty

Time: 7:50 - 8:30 (40 seconds)

We propose a five-step architecture process for dealing with uncertainty. First, identify uncertainties associated with the system. Second, for each uncertainty, determine if it is essential or accidental. Third, study the source of uncertainty—external sources are often accidental. Fourth, determine if a transformation can remove the uncertainty. Fifth, if so, apply fault-tolerance and related techniques. If not, you're dealing with essential uncertainty and need different approaches.

Slide 17: Analyze Uncertainties — Input

Time: 8:30 - 8:55 (25 seconds)

When analyzing input uncertainties, ask: Is it due to imprecise or incomplete specification? Is it due to variability in data—transmission issues, user errors, and so on? What can and cannot be known about input at runtime? These questions help you classify the uncertainty and choose appropriate tactics.

Slide 18: Analyze Uncertainties — Output

Time: 8:55 - 9:25 (30 seconds)

For output uncertainties, consider: What is the objective of the system? How do we know it meets the objective? Is there a known correct output? Is the correct output unknowable

but estimable? Can output be deterministically derived from inputs? Are there multiple correct outputs, and if so, is one preferable? These questions position you on the uncertainty spectrum.

Slide 19: Analyze Uncertainties — Transformation

Time: 9:25 - 9:45 (20 seconds)

For transformation uncertainties, ask: Is the relationship between input and output deterministic? If deterministic, is it tractable? These questions determine whether you need approximation algorithms, AI approaches, or can use traditional methods.

Slide 20: Case Study: Helicopter Rotor Balancing

Time: 9:45 - 10:15 (30 seconds)

Let me illustrate this with a real case study: helicopter rotor balancing. If the main rotor is not perfectly balanced, excess vibration occurs. This vibration causes wear, loss of power, and can compromise safety. The rotor is a complex system with multiple adjustment points, and the relationships between adjustments and outcomes are not straightforward.

Slide 21: Uncertainty Analysis — Input (Rotor)

Time: 10:15 - 10:40 (25 seconds)

For the input analysis: The input is known and precisely measured—vibration measurements, blade positions, adjustment points. User entry errors are external and accidental; we apply standard Fault Tolerance techniques to correct this accidental uncertainty. However, each helicopter has unique wear patterns—this is essential uncertainty that must be considered.

Slide 22: Uncertainty Analysis — Output (Rotor)

Time: 10:40 - 11:05 (25 seconds)

For output: The objective is measurable—change in vibration, measured in inches per second, or IPS. The desired output is a range—lower is better. The acceptable standard is 0.5 IPS. But is there a single solution? There may be different sequences of adjustments that achieve the goal, so we have some essential uncertainty here.

Slide 23: Uncertainty Analysis — Processing (Rotor)

Time: 11:05 - 11:30 (25 seconds)

For processing: Is the relationship between input and output deterministic? Initially, this was unknown. Combinations of adjustments appeared non-deterministic or even chaotic. The initial approach was to use a neural network to derive suggested solutions, treating this as essential uncertainty requiring AI-based methods.

Slide 24: Further Analysis (Rotor)

Time: 11:30 - 11:55 (25 seconds)

Further analysis revealed something interesting. Each helicopter has a unique wear profile. The initial neural network was trained on many helicopters, creating a generic approximation. But the adjustment and response history of testing reveals unique profiles. The intermediate result is that there exists a deterministic, repeatable sequence of adjustments with feedback loops. The details are proprietary, but this shows that some of what appeared essential was actually accidental, solvable with the right approach.

Slide 25: Relevant Results

Time: 11:55 - 12:20 (25 seconds)

The relevant results are impressive. The system reliably balances main rotors to 0.05 IPS—that's ten times better than the standard of 0.5 IPS. It's actively marketed to the US Army and other organizations. This demonstrates the importance of understanding essential versus accidental uncertainty—to choose the right architectural approach. Properly classifying uncertainty led to a much better solution.

Slide 26: Case Study: Online Bookstore

Time: 12:20 - 12:40 (20 seconds)

Let's look at another case study: an online bookstore. Initially, inputs and outputs were well-specified, with numerous accidental uncertainties like unreliable networks. The architecture was straightforward.

Slide 27: Bookstore Evolution — Related Books

Time: 12:40 - 13:05 (25 seconds)

But then they added a “related books” feature. Which books are related? This is an essential uncertainty—the output cannot be known beforehand, and there’s no single right answer. The approaches become probabilistic, requiring recommendation algorithms and similarity metrics.

Slide 28: Bookstore Evolution — Promotions

Time: 13:05 - 13:30 (25 seconds)

The evolution continued with using browsing history to promote products. This introduces even more uncertainty; the problem is less well-defined. This requires machine learning approaches and maps to a different position on the uncertainty spectrum. Recommendation engines are a popular, common solution for this type of problem.

Slide 29: References

Time: 13:30 - 13:45 (15 seconds)

Our work is published in HICSS 2026, and we also presented related work at the Intermountain Conference on Engineering, Technology and Computing in 2025. The full paper provides more detail on the framework and additional case studies.

Slide 30: Thank you!

Time: 13:45 - 15:00 (75 seconds)

Thank you for your attention. To summarize: We’ve presented a framework for understanding essential versus accidental uncertainty in software architecture. This distinction is crucial for selecting appropriate architectural approaches. Essential uncertainty requires probabilistic and AI-based methods, while accidental uncertainty can be addressed with traditional fault-tolerance techniques. The key is proper analysis and classification.

I’m happy to take questions now.

Notes for Practice:

- **Pacing:** Average 30 seconds per slide, but adjust based on complexity
- **Transitions:** Use phrases like “Let me illustrate,” “Moving along,” “The key insight is”
- **Emphasis:** Stress “essential” vs “accidental” throughout
- **Eye contact:** Look at audience, not just slides

- **Questions:** Pause after thought question slide to let it sink in
- **Case studies:** These are concrete examples—speak clearly and confidently
- **Time buffer:** Built in 15 seconds at end for questions; adjust if needed